MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (Autonomous)
Kokapet(Village), Gandipet, Hyderabad, Telangana – 500075. www.mgit.ac.in
UGC Autonomous | Affiliation to | AICTE approved | Accredited by A++ NAAC | NBA

MOTIVATE INNOVATE EMPOWER 26 YEARS

Pneumonia Detection: Pushing the Boundaries of Human Ability with Deep Learning

Project Supervisor

Mrs. U. Chaitanya,
Asst. Professor,
MGIT.

Project Member

Shahista Tamkeen
(20261A1249)

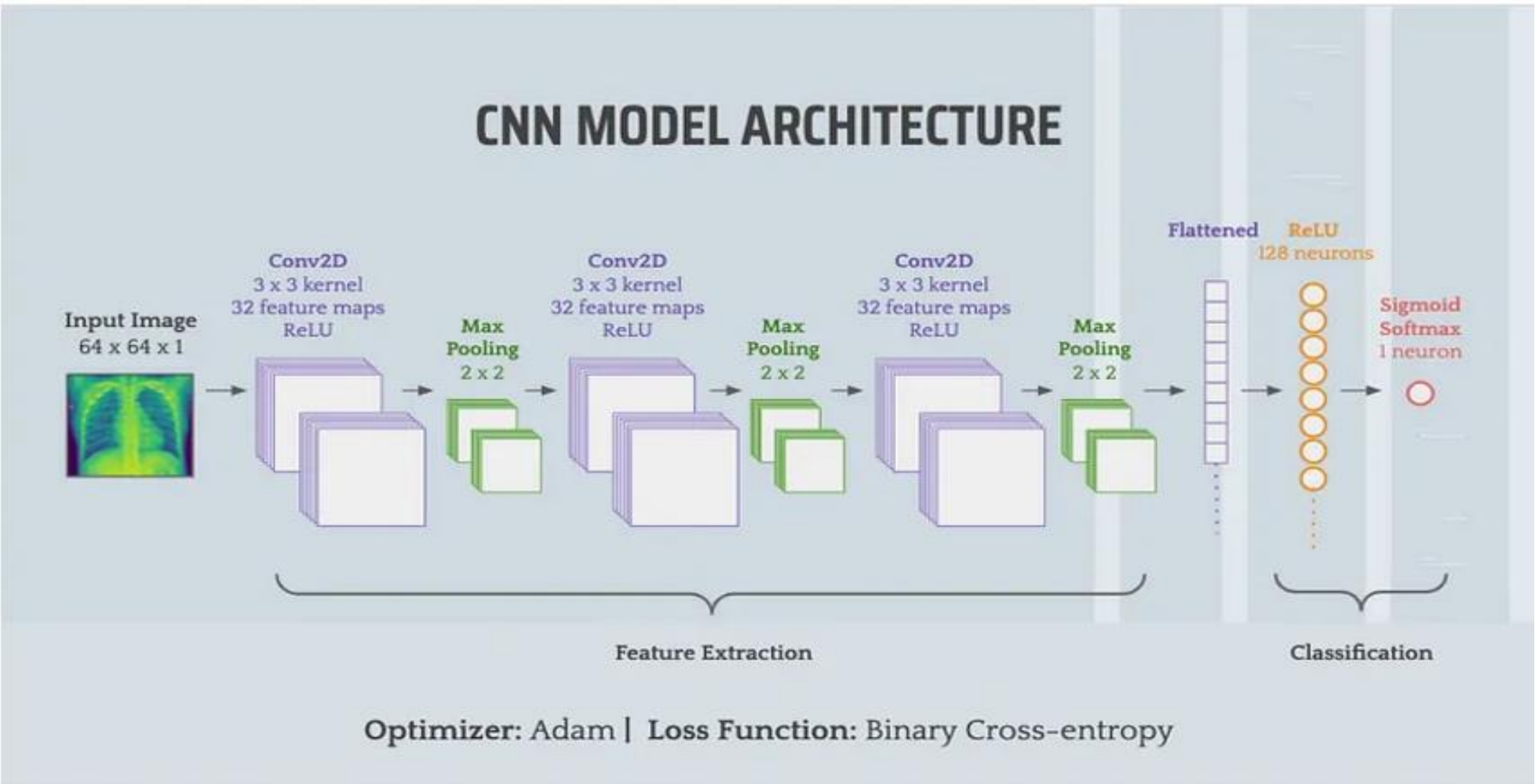
OUTLINE

1. Introduction
2. Model Architecture
3. Methodology
4. System Requirements
5. Coding
6. Testing
7. Deployment
8. Results
9. Conclusion
10. Q&A

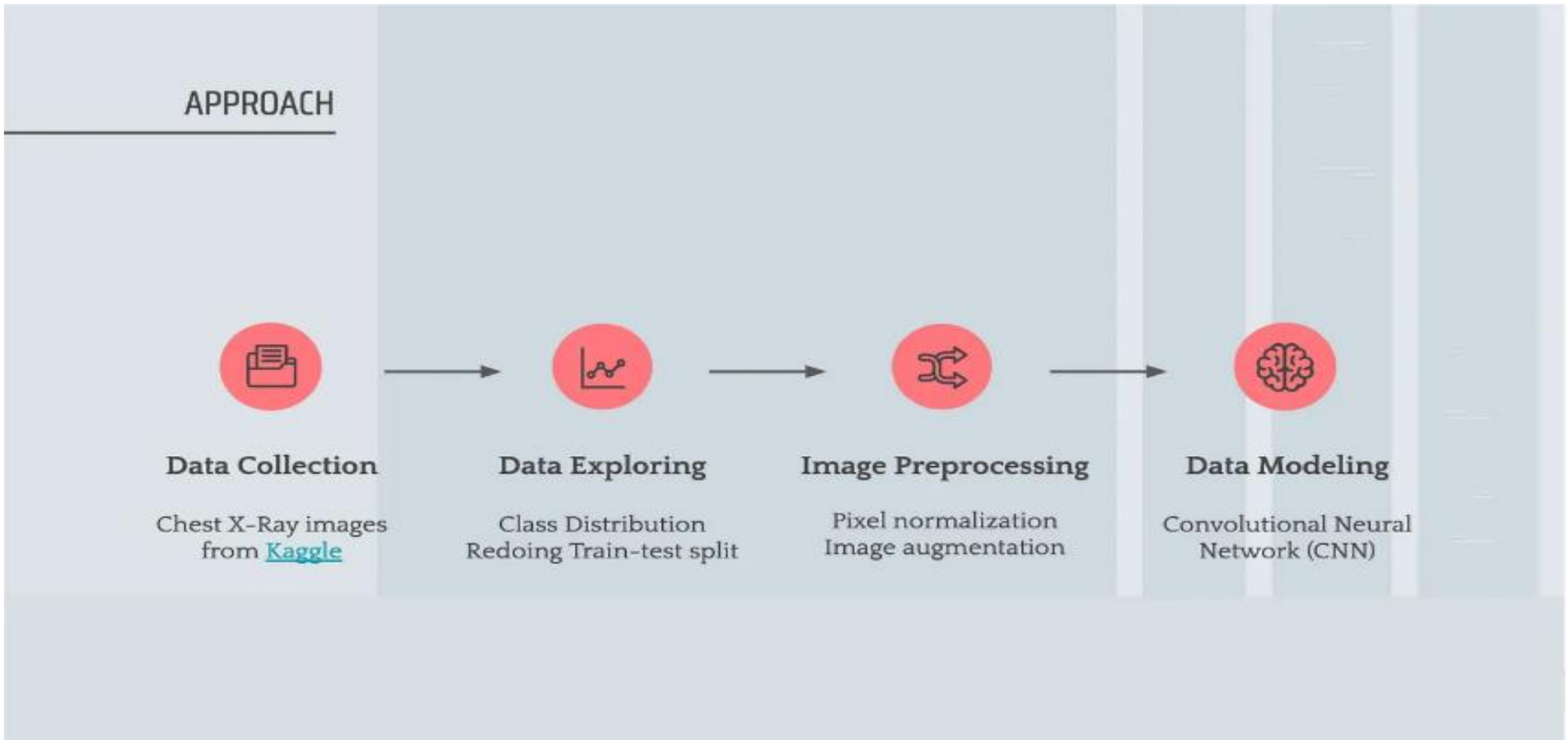
INTRODUCTION

- Pneumonia, a life-threatening lung infection, remains a global health challenge, especially for children and the elderly.
- Accurate and timely diagnosis using chest X-rays is vital but can be prone to human error and delays.
- This project harnesses the power of **deep learning** to revolutionize pneumonia detection.
- By leveraging **Convolutional Neural Networks (CNNs)**, we automate the analysis of chest X-ray images, ensuring faster and more accurate diagnoses.
- Our approach integrates advanced AI techniques with healthcare, aiming to enhance diagnostic precision, reduce human workload, and save lives. This project highlights the transformative role of AI in addressing critical medical challenges.

MODEL ARCHITECTURE



METHODOLOGY



METHODOLOGY

A systematic **approach** for building a machine learning model using chest X-ray images. It outlines the process in four key steps:

- **Data Collection**:
  - This step involves gathering chest X-ray images from Kaggle (a data science platform) as the primary dataset for the project.
- **Data Exploring**:
  - This phase focuses on analyzing the dataset, including checking the class distribution (e.g., normal vs. diseased cases) and reorganizing the train-test split to ensure balanced and reliable model evaluation.

METHODOLOGY

- **Image Preprocessing**:
  - In this step, the images are prepared for training by applying pixel normalization (to standardize pixel intensity values) and image augmentation (to artificially expand the dataset by flipping, rotating, or adjusting images).
- **Data Modeling**:
  - Finally, a Convolutional Neural Network (CNN), a type of deep learning model specifically designed for image data, is used to build and train the predictive model.

SYSTEM REQUIREMENTS

SOFTWARE REQUIREMENTS

Operating system	: Windows 10
programming language	: Python
Tools	: Google Collab,Visual Studio Code

HARDWARE REQUIREMENTS

Processor	: Intel Core i7
RAM	: minimum 8 GB

CODING

Import necessary libraries

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
from sklearn.metrics import auc
import keras
import tensorflow as tf
from keras import backend as K
from keras import metrics
from keras.regularizers import l2
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization, Input
from keras.preprocessing.image import ImageDataGenerator
from keras.metrics import AUC
from keras.optimizers import Adam
from keras.callbacks import EarlyStopping
from sklearn.metrics import classification_report, confusion_matrix
```

CODING

Train the model

```
cnn = Sequential()
cnn.add(InputLayer(input_shape=(hyper_dimension, hyper_dimension, hyper_channels)))

cnn.add(Conv2D(filters=32, kernel_size=3, activation='relu'))
cnn.add(MaxPooling2D(pool_size=(2,2)))

cnn.add(Conv2D(filters=32, kernel_size=3, activation='relu'))
cnn.add(MaxPooling2D(pool_size=(2,2)))

cnn.add(Conv2D(filters=32, kernel_size=3, activation='relu'))
cnn.add(MaxPooling2D(pool_size=(2,2)))

cnn.add(Flatten())

cnn.add(Dense(activation='relu', units=128))
cnn.add(Dense(activation='sigmoid', units=1))

cnn.compile(optimizer= 'adam', loss='binary_crossentropy', metrics=[AUC()])
cnn_model = cnn.fit_generator(train_generator,
                              steps_per_epoch = len(train_generator),
                              epochs = 200,
                              validation_data = val_generator,
                              validation_steps = len(val_generator),
                              verbose=2)
```

TESTING

- **Test Case 1**: Validate that the ImageDataGenerator correctly loads images from the specified directories (train, val, and test paths).

  Expected Result: All images are loaded, and data augmentation (for the training set) works correctly.

- **Test Case 2**: Ensure the image resizing, normalization, and color mode (grayscale) are applied properly.

  Expected Result: All images are resized to (64x64) and normalized (pixel values between 0 and 1).

- **Test Case 5**: Check if the model architecture is created successfully with the correct number of layers, filters, and neurons.

TESTING

Expected Result: The model includes 3 convolutional layers, max-pooling layers, a flattening layer, and two dense layers.

- **Test Case 5**: Check if the model architecture is created successfully with the correct number of layers, filters, and neurons.

  Expected Result: The model includes 3 convolutional layers, max-pooling layers, a flattening layer, and two dense layers.

- **Test Case 7**: Ensure that the model trains for the specified number of epochs (200) and provides validation metrics during training.

  Expected Result: The model completes training with validation loss and AUC displayed.

TESTING

- **Test Case 8**: Verify that the training process respects the random seed for reproducibility.

  Expected Result: The same training results are obtained when running the code multiple times with the same seed.

DEPLOYMENT

- Using flask and programming language in python,we have run the web application.

Tools and Technologies:

TensorFlow/Keras: Used for training, saving, and loading the model for deployment.

Flask/FastAPI/Django: Lightweight web frameworks for creating REST APIs to serve the model predictions.

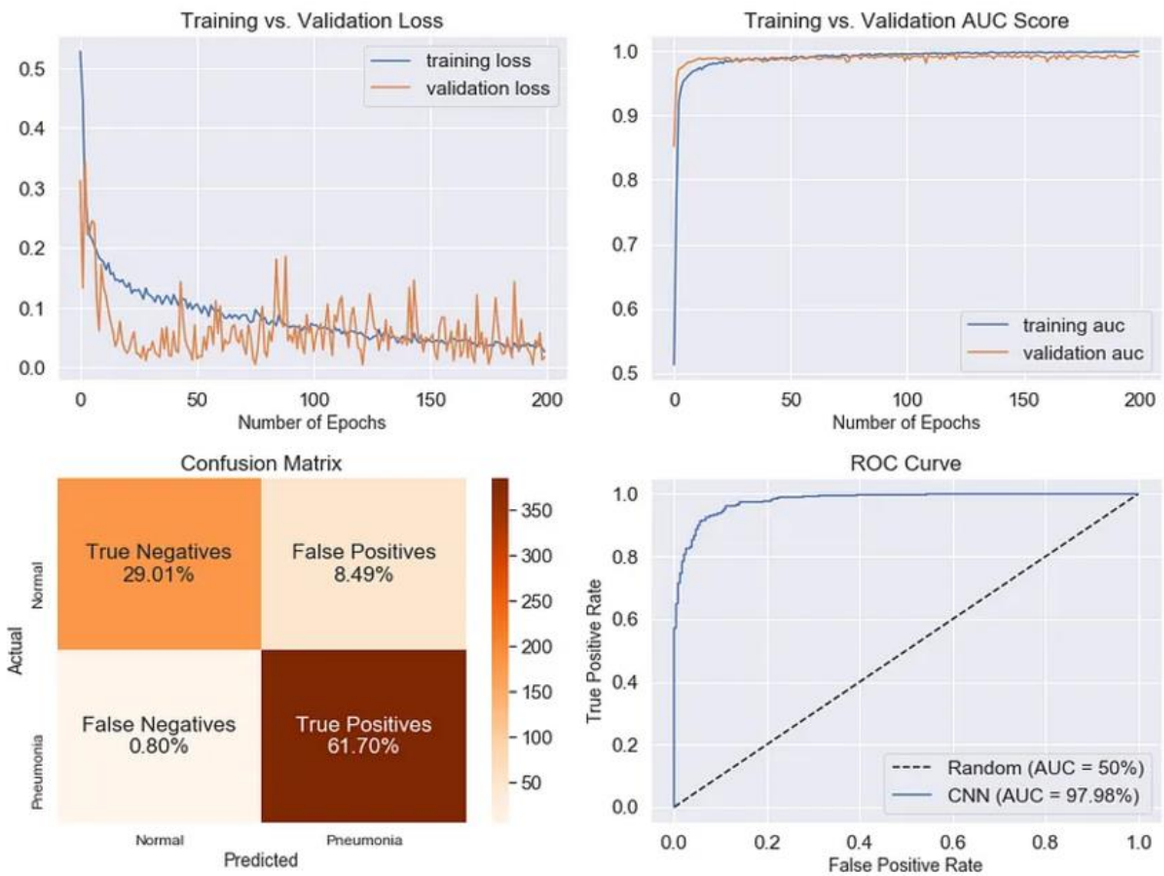
TensorFlow Lite: For deploying the model on mobile or embedded devices.

AWS (Amazon Web Services): Services like EC2 for hosting, S3 for storage, and SageMaker for deploying models.

Results

Output results via metrics and charts

```
[Summary Statistics]
Accuracy = 90.71% | Precision = 87.90% | Recall = 98.72% |
Specificity = 77.35% | F1 Score = 93.00%
```



Metrics and charts of the final model

CONCLUSION

The pneumonia detection project showcases the potential of deep learning in healthcare diagnostics. By using CNNs and chest X-ray images, it provides a fast, reliable solution for identifying pneumonia.

The model's efficient pipeline and deployment scalability make it valuable for real-world applications. Future improvements can enhance its accuracy and impact, revolutionizing early disease detection and medical decision-making.

THANK YOU